

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmangl10_selfie_data_path = kagglehub.dataset_download('rehmangl10/selfie-data')
rehmangl10_facemeshes_path = kagglehub.dataset_download('rehmangl10/facemeshes')
rehmangl10_smilefces_path = kagglehub.dataset_download('rehmangl10/smilefces')
rehmangl10_smilevideo_path = kagglehub.dataset_download('rehmangl10/smilevideo')
rehmangl10_frown_faces_path = kagglehub.dataset_download('rehmangl10/frown-faces')
rehmangl10_surprised_dataset_path = kagglehub.dataset_download('rehmangl10/surprised-dataset')
rehmangl10_normalfaces_path = kagglehub.dataset_download('rehmangl10/normalfaces')
rehmangl10_saddataset_path = kagglehub.dataset_download('rehmangl10/saddataset')
rehmangl10_3dmeshes_path = kagglehub.dataset_download('rehmangl10/3dmeshes')

print('Data source import complete.')
```

```
!pip install mediapipe
!pip install trimesh
```

[Show hidden output](#)

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load
```

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

```
import torch
from pathlib import Path
from PIL import Image
import mediapipe as mp
import json
import cv2
import os
import shutil
import trimesh
from scipy.spatial import cKDTree
```

[Show hidden output](#)

```
from pathlib import Path
import shutil
```

```
print(torch.__version__)
print(np.__version__)
print(mp.__version__)
print(cv2.__version__)
!python --version
```

```
2.6.0+cu124
1.26.4
0.10.21
4.11.0
Python 3.11.13
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype = torch.float16
```

```
else:
    device ="cpu"
    dtype = torch.float32

print("the device is :",device)
print("the Data type is : " , dtype)
```

```
the device is : cpu
the Data type is : torch.float32
```

```
# move the the files from the input to working directory
src_path = Path("/kaggle/input/normalfaces")
dest_path = Path("/kaggle/working/normalfaces")
dest_path.mkdir(parents=True , exist_ok=True)
for index , file in enumerate(src_path.iterdir()):
    shutil.copy(file , dest_path / f"selfie{index}.jpeg")
```

```
# move the the files from the input to working directory
src_path = Path("/kaggle/input/selfie-data")
dest_path = Path("/kaggle/working/face_pics")
dest_path.mkdir(parents=True , exist_ok=True)
for index , file in enumerate(src_path.iterdir()):
    shutil.copy(file , dest_path / f"selfie{index}.jpeg")
```

```
# move the the files from the input to working directory move the meshes
src_path = Path("/kaggle/input/3dmeshes")
dest_path = Path("/kaggle/working/real3dmeshes")
dest_path.mkdir(parents=True , exist_ok=True)
for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
# move the the files from the input to working directory move the meshes
src_path = Path("/kaggle/input/smilevideo")
dest_path = Path("/kaggle/working/smilevideo")
dest_path.mkdir(parents=True , exist_ok=True)
for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
# move the the files from the input to working directory move the meshes
src_path = Path("/kaggle/input/facemeshes")
dest_path = Path("/kaggle/working/face_meshes")
dest_path.mkdir(parents=True , exist_ok=True)
for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
# move the the files from the input to working directory move the meshes
src_path = Path("/kaggle/input/frown-faces")
dest_path = Path("/kaggle/working/frown-faces")
dest_path.mkdir(parents=True , exist_ok=True)
for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
# move the the files from the input to working directory move the meshes
src_path = Path("/kaggle/input/surprised-dataset")
dest_path = Path("/kaggle/working/surprised-dataset")
dest_path.mkdir(parents=True , exist_ok=True)
for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
image_path = "/kaggle/working/surprised-dataset/sur3.jpg"
```

```
Image.open(image_path)
```



```
#now our task is to intialize a face mesh mediapipe
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(static_image_mode=True , max_num_faces=1 ,refine_landmarks=True)
```

```
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
W0000 00:00:1754540500.049818      120 inference_feedback_manager.cc:114] Feedback manager requires a model with a signature
W0000 00:00:1754540500.078398      118 inference_feedback_manager.cc:114] Feedback manager requires a model with a signature
```

```
# load the selfie
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(static_image_mode=True , max_num_faces=1 ,refine_landmarks=True)
selfie_path = Path(image_path)
image = cv2.imread(str(selfie_path))
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # got the numpy array

#process the image
results = face_mesh.process(image_rgb)

# now we will see the results , landmarks
landmarks = []
if results.multi_face_landmarks:
    for lm in results.multi_face_landmarks[0].landmark:
        x= lm.x * image.shape[1]
        y = lm.y * image.shape[0]
        z= lm.z * image.shape[1] # scale z to image width
        landmarks.append([x,y,z])

landmarks =np.array(landmarks)
#Image.open("/kaggle/working/landmarks_visualize.jpg")
landmark_xy = [[x, y] for x, y, z in landmarks]
landmark_xy = np.array(landmark_xy)
print(landmark_xy.shape)
```

```
(478, 2)
W0000 00:00:1754540501.445642      122 inference_feedback_manager.cc:114] Feedback manager requires a model with a signature
W0000 00:00:1754540501.473929      123 inference_feedback_manager.cc:114] Feedback manager requires a model with a signature
W0000 00:00:1754540501.487048      123 landmark_projection_calculator.cc:186] Using NORM_RECT without IMAGE_DIMENSION
```

```
#we have the expression dictionary we needed
expression_landmarks = {
    "smile": {
        "left_mouth_corner": 61,
        "right_mouth_corner": 291,
        "upper_lip": 13,
        "lower_lip": 14
    },
    "frown": {
        "left_inner_brow": 107,
        "left_mid_brow": 63,
        "left_outer_brow": 105,
```

```

        "right_inner_brow": 336,
        "right_mid_brow": 296,
        "right_outer_brow": 334,
        "left_eye_top": 159,
        "right_eye_top": 386
    },
    "surprise": {
        "upper_lip": 13,
        "lower_lip": 14,
        "left_eye_top": 159,
        "left_eye_bottom": 145,
        "right_eye_top": 386,
        "right_eye_bottom": 374
    },
    "neutral": {
        "left_mouth_corner": 61,
        "right_mouth_corner": 291,
        "upper_lip": 13,
        "lower_lip": 14,
        "left_eye_top": 159,
        "left_eye_bottom": 145,
        "right_eye_top": 386,
        "right_eye_bottom": 374,
        "left_inner_brow": 70,
        "right_inner_brow": 336
    },
    "sad": {
        "left_mouth_corner": 61,
        "right_mouth_corner": 291,
        "lower_lip": 14,
        "left_eye_outer_corner": 33,
        "right_eye_outer_corner": 263,
        "left_inner_brow": 70,
        "right_inner_brow": 336
    },
    "angry": {
        "left_inner_brow": 70,
        "right_inner_brow": 336,
        "left_eye_top": 159,
        "left_eye_bottom": 145,
        "right_eye_top": 386,
        "right_eye_bottom": 374,
        "left_mouth_corner": 61,
        "right_mouth_corner": 291
    }
}

```

```

def detect_smile(landmark):
    # Calculate distances
    mouth_width = np.linalg.norm(landmark[61] - landmark[291])
    mouth_height = np.linalg.norm(landmark[13] - landmark[14])

    # Reference: face width (from cheek to cheek)
    face_width = np.linalg.norm(landmark[234] - landmark[454])

    # Avoid division by zero
    if face_width == 0:
        return False

    # Ratios
    mouth_width_ratio = mouth_width / face_width
    mouth_height_ratio = mouth_height / face_width

    # Heuristic thresholds (can be tuned)
    if mouth_width_ratio > 0.40 and mouth_height_ratio > 0.05:
        return True
    else:
        return False

```

```

# Video I/O
input_path = "/kaggle/working/smilevideo/smilevideo.mp4"
output_path = "output_smile_detected.mp4"

```

```

cap = cv2.VideoCapture(input_path)

# Get video properties
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(output_path, fourcc, fps, (width, height))

while cap.isOpened():
    success, frame = cap.read()
    if not success:
        break

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = face_mesh.process(rgb)

    if results.multi_face_landmarks:
        face = results.multi_face_landmarks[0].landmark

        # Convert to pixel-based 2D landmark array
        landmark = np.array([[lm.x * width, lm.y * height] for lm in face])

        # Detect smile
        is_smile = detect_smile(landmark)
        label = "😊 Smile" if is_smile else "😞 Not Smiling"

        # Draw label
        cv2.putText(frame, label, (30, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 255, 0) if is_smile else (0, 0, 255),
                    out.write(frame)

cap.release()
out.release()

```

```

def detect_frown(landmark):
    # Inner brow positions
    left_brow = landmark[107]
    right_brow = landmark[336]

    # Eye top positions (used as reference height)
    left_eye = landmark[159]
    right_eye = landmark[386]

    # Vertical drop of inner brows (how low brows are compared to eyes)
    left_drop = left_eye[1] - left_brow[1]
    right_drop = right_eye[1] - right_brow[1]

    # Horizontal distance between brows (to detect contraction)
    brow_distance = np.linalg.norm(left_brow - right_brow)

    # Face width as reference scale
    face_width = np.linalg.norm(landmark[234] - landmark[454])
    if face_width == 0:
        return False

    # Normalize values
    drop_ratio = (left_drop + right_drop) / 2 / face_width
    brow_contraction_ratio = brow_distance / face_width
    print(drop_ratio)
    print(brow_contraction_ratio)
    # Heuristic thresholds (tune as needed)
    is_lowered = drop_ratio < 0.025 # try lower threshold
    is_contracted = brow_contraction_ratio < 0.25 # slightly stricter

    return is_lowered and is_contracted

```

```

def detect_surprise(landmark):
    lip_distance = np.linalg.norm(landmark[13] - landmark[14])
    left_eye_dist = np.linalg.norm(landmark[159] - landmark[145])
    right_eye_dist = np.linalg.norm(landmark[386] - landmark[374])

```

```

face_height = np.linalg.norm(landmark[10] - landmark[152])

if face_height < 30: # pixels this means the face is too small and far from the camera
    return False

norm_lip_dist = lip_distance / face_height
norm_eye_dist = (left_eye_dist + right_eye_dist) / 2 / face_height
print(norm_lip_dist)
print(norm_eye_dist)

lip_open = norm_lip_dist > 0.10
eye_open = norm_eye_dist > 0.05

return lip_open and eye_open

```

```

def detect_neutral(landmark):
    face_width = np.linalg.norm(landmark[234] - landmark[454])
    face_height = np.linalg.norm(landmark[10] - landmark[152])

    if face_width < 10 or face_height < 10:
        return False

    # Distances
    mouth_width = np.linalg.norm(landmark[61] - landmark[291])
    mouth_opening = np.linalg.norm(landmark[13] - landmark[14])
    left_eye_opening = np.linalg.norm(landmark[145] - landmark[159])
    right_eye_opening = np.linalg.norm(landmark[374] - landmark[386])
    left_brow_drop = landmark[159][1] - landmark[107][1]
    right_brow_drop = landmark[386][1] - landmark[336][1]

    # Normalized
    norm_mouth_width = mouth_width / face_width
    norm_mouth_opening = mouth_opening
    norm_eye_opening = (left_eye_opening + right_eye_opening) / 2 / face_height
    norm_brow_drop = (left_brow_drop + right_brow_drop) / 2 / face_height

    # Debug log
    print("Normalized Mouth Width:", round(norm_mouth_width, 3))
    print("Normalized Mouth Opening:", round(norm_mouth_opening, 3))
    print("Normalized Eye Opening:", round(norm_eye_opening, 3))
    print("Normalized Brow-to-Eye Drop:", round(norm_brow_drop, 3))
    raw_open = np.linalg.norm(np.array(landmark[13]) - np.array(landmark[14]))
    print("Raw mouth opening:", raw_open)

    # Thresholds
    mouth_width_ok = norm_mouth_width < 0.40
    mouth_closed = norm_mouth_opening < 0.6
    eyes_relaxed = norm_eye_opening < 0.07
    brows_normal = norm_brow_drop < 0.08

    return mouth_width_ok and mouth_closed and eyes_relaxed and brows_normal

```

```

def detect_sad(landmark):
    face_height = np.linalg.norm(landmark[10] - landmark[152])
    if face_height < 10:
        return False

    # Brow logic (same as before)
    left_brow = landmark[107]
    right_brow = landmark[336]
    left_eye = landmark[33]
    right_eye = landmark[263]

    left_brow_rise = left_brow[1] - left_eye[1]
    right_brow_rise = right_brow[1] - right_eye[1]
    avg_brow_rise = (left_brow_rise + right_brow_rise) / 2
    norm_brow_rise = avg_brow_rise / face_height

    # Mouth corner slope
    left_corner = landmark[61]
    right_corner = landmark[291]
    mouth_slope = (right_corner[1] - left_corner[1]) / (right_corner[0] - left_corner[0] + 1e-6) # avoid div by 0

```

```
# Debug
print("Mouth Slope:", round(mouth_slope, 3))
print("Brow Rise:", round(norm_brow_rise, 3))

is_brow_raised = norm_brow_rise < -0.11
is_mouth_tilted_downward = mouth_slope > 0.080 # > 0 = right lower than left (downward tilt)

return is_brow_raised and is_mouth_tilted_downward
```

```
def get_smile_weight(landmarks):
    mouth_width = np.linalg.norm(landmarks[61] - landmarks[291])
    mouth_height = np.linalg.norm(landmarks[13] - landmarks[14])
    face_width = np.linalg.norm(landmarks[234] - landmarks[454])

    if face_width == 0:
        return 0.0

    mouth_width_ratio = mouth_width / face_width
    mouth_height_ratio = mouth_height / face_width

    min_height_ratio = 0.02
    max_height_ratio = 0.08

    weight = (mouth_height_ratio - min_height_ratio) / (max_height_ratio - min_height_ratio)
    return float(np.clip(weight, 0.0, 1.0))

def get_surprise_weight(landmarks):
    lip_distance = np.linalg.norm(landmarks[13] - landmarks[14])
    left_eye_dist = np.linalg.norm(landmarks[159] - landmarks[145])
    right_eye_dist = np.linalg.norm(landmarks[386] - landmarks[374])
    face_height = np.linalg.norm(landmarks[10] - landmarks[152])

    if face_height < 30:
        return 0.0

    norm_lip_dist = lip_distance / face_height
    norm_eye_dist = (left_eye_dist + right_eye_dist) / (2 * face_height)

    lip_range = (0.05, 0.20)
    eye_range = (0.03, 0.10)

    lip_weight = (norm_lip_dist - lip_range[0]) / (lip_range[1] - lip_range[0])
    eye_weight = (norm_eye_dist - eye_range[0]) / (eye_range[1] - eye_range[0])

    lip_weight = np.clip(lip_weight, 0.0, 1.0)
    eye_weight = np.clip(eye_weight, 0.0, 1.0)

    return float((lip_weight + eye_weight) / 2)

def get_sad_weight(landmarks):
    face_height = np.linalg.norm(landmarks[10] - landmarks[152])
    if face_height < 10:
        return 0.0

    left_brow = landmarks[107]
    right_brow = landmarks[336]
    left_eye = landmarks[33]
    right_eye = landmarks[263]

    left_brow_rise = left_brow[1] - left_eye[1]
    right_brow_rise = right_brow[1] - right_eye[1]
    avg_brow_rise = (left_brow_rise + right_brow_rise) / 2
    norm_brow_rise = avg_brow_rise / face_height

    left_corner = landmarks[61]
    right_corner = landmarks[291]
    mouth_slope = (right_corner[1] - left_corner[1]) / (right_corner[0] - left_corner[0] + 1e-6)

    brow_min, brow_max = -0.20, -0.05
    brow_weight = (norm_brow_rise - brow_max) / (brow_min - brow_max)
    brow_weight = np.clip(brow_weight, 0.0, 1.0)

    slope_min, slope_max = 0.03, 0.15
```

```

mouth_weight = (mouth_slope - slope_min) / (slope_max - slope_min)
mouth_weight = np.clip(mouth_weight, 0.0, 1.0)

return float((brow_weight + mouth_weight) / 2)

```

```

# === Define Paths ===
neutral_path = "/kaggle/working/real3dmeshes/head.obj"
expression_meshes = {
    "smile": "/kaggle/working/real3dmeshes/smileobj.obj",
    "sad": "/kaggle/working/real3dmeshes/sadhead.obj",
    "surprise": "/kaggle/working/real3dmeshes/surprisehead.obj"
}
save_dir = "/kaggle/working/deltas"
os.makedirs(save_dir, exist_ok=True)

# === Load Neutral Mesh ===
neutral_mesh = trimesh.load(neutral_path, process=False)
neutral_vertices = np.array(neutral_mesh.vertices)

# === Loop Over Expression Meshes ===
for name, path in expression_meshes.items():
    print(f"\n🔄 Processing {name}...")

    # Load expression mesh
    expr_mesh = trimesh.load(path, process=False)
    expr_vertices = np.array(expr_mesh.vertices)

    # Topology check
    if expr_vertices.shape != neutral_vertices.shape:
        print(f"❌ Skipping {name}: vertex count mismatch.")
        continue

    # Compute delta
    delta = expr_vertices - neutral_vertices

    # Save delta
    save_path = os.path.join(save_dir, f"{name}.npz")
    np.save(save_path, delta)
    print(f"✅ Saved {name} delta to: {save_path}")

```

```

🔄 Processing smile...
✅ Saved smile delta to: /kaggle/working/deltas/smile.npz

🔄 Processing sad...
✅ Saved sad delta to: /kaggle/working/deltas/sad.npz

🔄 Processing surprise...
✅ Saved surprise delta to: /kaggle/working/deltas/surprise.npz

```

```

# Load neutral face
neutral_mesh = trimesh.load("/kaggle/working/real3dmeshes/head.obj", process=False)
neutral_vertices = neutral_mesh.vertices.copy()
faces = neutral_mesh.faces

# Your weights
expression_weights = {
    "smile": get_smile_weight(landmark_xy),
    "sad": get_sad_weight(landmark_xy),
    "surprise": get_surprise_weight(landmark_xy)
}

# Apply deltas
morphed_vertices = neutral_vertices.copy()
for expr, weight in expression_weights.items():
    delta_path = f"/kaggle/working/deltas/{expr}.npz"
    if os.path.exists(delta_path):
        delta = np.load(delta_path)
        morphed_vertices += weight * delta
    else:
        print(f"⚠️ No delta found for {expr}")

# Export result

```



```
morphed_mesh = trimesh.Trimesh(vertices=morphed_vertices, faces=faces, process=False)
morphed_mesh.export("/kaggle/working/morphed_face_sur.obj")
print("✅ Done! Morphed face saved.")
```

✅ Done! Morphed face saved.